

How Backpropagation is Implemented: Building a Mini Autograd Engine from Scratch

Further Study 01 — Deep Learning II

Jihun Jang
02521122
UST / ETRI
April 2026

1. Research Background

Backpropagation is the fundamental algorithm behind training neural networks. There are two primary approaches to computing gradients: the **analytical approach**, where gradients are derived manually using calculus, and the **computational graph approach**, where gradients are computed automatically by traversing a directed acyclic graph (DAG) of operations.

Modern deep learning frameworks such as PyTorch adopt the computational graph approach. PyTorch builds a **dynamic computational graph** during the forward pass, recording each operation and its inputs. During the backward pass, it traverses this graph in reverse topological order, applying the chain rule to propagate gradients from the loss back to every parameter.

While PyTorch's *autograd* module handles this process automatically, understanding its internal mechanics is crucial for debugging gradient flow issues, implementing custom operations, and reasoning about training behavior in complex architectures such as GANs or Vision-Language-Action Models (VLAMs).

Motivation. The goal of this study is not merely to replicate PyTorch's autograd, but to **understand the code-level flow of computational graph construction and execution**. By implementing a minimal autograd engine from scratch using only NumPy, we aim to internalize: (1) how the graph is dynamically built during forward computation, (2) how topological sorting ensures correct gradient propagation order, and (3) how gradient accumulation handles multiple-path contributions.

2. Theoretical Development

2.1 Computational Graph Structure

A computational graph represents a mathematical expression as a DAG. Each **node** represents an individual operation (addition, multiplication, etc.), and each **edge** represents the flow of data (intermediate values) between operations. Unlike a neural network diagram where a neuron is the basic unit, the computational graph decomposes each neuron into its constituent primitive operations.

For example, a single neuron computing $y = w \cdot x + b$ is represented as two nodes: a multiplication node ($*$) and an addition node ($+$). This decomposition is the key insight — each primitive operation has a simple, known local gradient.

2.2 Forward Pass: Value Computation + Graph Construction

During the forward pass, two things happen simultaneously: (1) the numerical output is computed, and (2) the graph topology is recorded. Each result node stores references to its parent nodes (`_prev`) and a closure (`_backward`) that encodes how to distribute gradients back to those parents. This is the essence of **dynamic graph construction** — the graph is built on-the-fly as operations execute.

2.3 Backward Pass: Topological Sort + Chain Rule

The backward pass computes gradients by traversing the graph in **reverse topological order**. Topological sorting ensures that when we process a node, all nodes that depend on it have already propagated their gradients. This is implemented via a DFS post-order traversal:

1. Start from the loss node and perform DFS, visiting children (parents in the computation) first.
2. Append each node to the ordering *after* all its children have been visited.
3. Reverse the ordering to get the backward execution sequence.
4. Set `loss.grad = 1.0`, then call each node's `_backward()` in this reversed order.

2.4 Local Gradient Patterns

Operation	Forward	Local Gradient	Pattern Name
$c = a + b$	$c = a + b$	$dc/da = 1, dc/db = 1$	Gradient Distributor
$c = a * b$	$c = a * b$	$dc/da = b, dc/db = a$	Swap Multiplier
$c = a ** n$	$c = a^n$	$dc/da = n * a^{(n-1)}$	Power Rule
$c = \text{relu}(a)$	$\max(0, a)$	$dc/da = 1 \text{ if } a > 0 \text{ else } 0$	Gradient Router
$c = \text{sigmoid}(a)$	$1/(1+e^{-a})$	$dc/da = c*(1-c)$	Sigmoid Derivative

Table 1. Local gradient patterns for primitive operations.

2.5 Multiple Path Contribution

When a variable is used in multiple operations (fan-out), it receives gradient contributions from each path. Mathematically, this follows from the multivariate chain rule. In code, this is handled by **accumulating** gradients with += rather than assigning with =. For example, if $f = a*a + a*b$ where $a=3$, $b=5$, then $df/da = 2a + b = 11$. The gradient 11 is the sum of contributions from the $a*a$ path ($2a=6$) and the $a*b$ path ($b=5$).

2.6 Correspondence with PyTorch Autograd

Our Implementation	PyTorch Equivalent	Role
Value.data	tensor.data	Stores the numerical value
Value.grad	tensor.grad	Accumulated gradient
Value._prev	grad_fn.next_functions	References to parent nodes
Value._backward	grad_fn (e.g., MulBackward0)	Backward function closure
build_topo (DFS)	Engine::execute()	Topological ordering for BP

Table 2. Mapping between our autograd engine and PyTorch internals.

3. Programming Verification

3.1 Value Class Implementation

The core of our autograd engine is the **Value** class. Each instance holds three essential pieces of information: *data* (the forward-pass result), *grad* (the accumulated gradient), and *_prev* (references to parent nodes) along with a *_backward* closure.

```
class Value:
    def __init__(self, data):
        self.data = data
        self.grad = 0.0
        self._prev = set()
        self._backward = lambda: None

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data)
        out._prev = {self, other}
        def _backward():
            self.grad += 1.0 * out.grad    # distributor
            other.grad += 1.0 * out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data)
        out._prev = {self, other}
        def _backward():
            self.grad += other.data * out.grad    # swap
            other.grad += self.data * out.grad
        out._backward = _backward
        return out
```

3.2 Additional Operations

We extended Value with pow, relu, sigmoid, negation, and subtraction. Each follows the same pattern: compute forward result, store parent references, define *_backward* closure with the appropriate local gradient.

```
def __pow__(self, n):
    out = Value(self.data ** n)
    out._prev = {self}
    def _backward():
        self.grad += (n * self.data ** (n-1)) * out.grad
    out._backward = _backward
    return out

def relu(self):
    out = Value(max(0, self.data))
    out._prev = {self}
    def _backward():
        self.grad += (1.0 if self.data > 0 else 0.0) * out.grad
    out._backward = _backward
    return out
```

3.3 Backward with Topological Sort

```
def backward(self):
    topo = []
    visited = set()
    def build_topo(node):
        if node not in visited:
            visited.add(node)
            for child in node._prev:
                build_topo(child)
            topo.append(node)
    build_topo(self)
    self.grad = 1.0
    for node in reversed(topo):
        node._backward()
```

3.4 MLP Construction

Using the Value class, we built Neuron, Layer, and MLP classes. Every arithmetic operation on Value objects automatically constructs the computational graph. The output layer omits activation to allow unrestricted output range for regression.

```
class Neuron:
    def __init__(self, nin, activation=True):
        self.w = [Value(uniform(-1,1)) for _ in range(nin)]
        self.b = Value(0.0)
        self.activation = activation
    def __call__(self, x):
        ws = self.b
        for wi, xi in zip(self.w, x):
            ws = ws + wi * xi
        return ws.relu() if self.activation else ws

class MLP:
    def __init__(self, nin, nouts):
        sz = [nin] + nouts
        self.layers = [Layer(sz[i], sz[i+1],
                             activation=(i != len(nouts)-1))
                       for i in range(len(nouts))]
    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x
```

3.5 Experiment: XOR Learning

We trained an MLP(2, [8, 4, 1]) on the XOR problem. The training loop follows the standard gradient descent procedure: forward pass, loss computation (MSE), gradient zeroing, backward pass, and parameter update. The model converged to near-zero loss within ~150 epochs.

```
model = MLP(2, [8, 4, 1])
lr = 0.05; epochs = 1000

for epoch in range(epochs):
    preds = [model(x) for x in xs]
    loss = Value(0)
    for pred, y in zip(preds, ys):
        loss = loss + (pred - y) ** 2
    for p in model.parameters():
        p.grad = 0.0 # zero grad
    loss.backward() # backpropagation
    for p in model.parameters():
        p.data -= lr * p.grad # gradient descent
```

XOR Training Results:

Input	Target	Prediction
[0, 0]	0	0.0000
[0, 1]	1	1.0000
[1, 0]	1	1.0000
[1, 1]	0	0.0000

Table 3. XOR predictions after training (loss converged to ~ 0).

3.6 PyTorch Verification

To validate correctness, we compared gradients from our engine against PyTorch's autograd on identical computations. All four test cases produced matching results:

Test Case	Variable	Ours	PyTorch	Match
$e = (a \cdot (a+b))^2$ $a=2, b=3$	a.grad	140.0	140.0	Y
	b.grad	40.0	40.0	Y
$w = \text{relu}(x+y) \cdot 2$ $x=-3, y=4$	x.grad	2.0	2.0	Y
	y.grad	2.0	2.0	Y
$c = \text{sigmoid}(a) \cdot 3$ $a=1.5$	a.grad	0.4474	0.4474	Y
$f = a \cdot a + a \cdot b$ $a=3, b=5$	a.grad	11.0	11.0	Y
	b.grad	3.0	3.0	Y

Table 4. Gradient comparison: our autograd vs PyTorch autograd.

4. Conclusion

By building a minimal autograd engine from scratch, we gained deep insight into how backpropagation works at the code level. The key findings are:

Computational graph as a design pattern. The separation of forward computation and backward gradient flow through a DAG provides modularity and extensibility. Adding a new operation requires only defining its forward computation and local gradient — the rest is handled by the graph infrastructure.

Closures enable deferred execution. The `_backward` function, defined at forward time but executed at backward time, captures the forward-pass context via Python closures. This is the mechanism that makes dynamic computational graphs possible.

Topological sort guarantees correctness. DFS post-order traversal ensures that upstream gradients are fully accumulated before being propagated further. Without this ordering, multiple-path contributions would produce incorrect results.

Practical lessons from XOR training. We encountered the dead ReLU problem when network depth and input scale were mismatched, and observed sensitivity to weight initialization. Removing activation from the output layer was necessary for regression tasks. These are the same issues practitioners face with full-scale frameworks.

PyTorch autograd as a production-grade extension. While our engine implements the core algorithm faithfully (verified by matching PyTorch's gradients exactly), PyTorch adds optimizations such as in-place operation tracking, gradient checkpointing for memory efficiency, and C++ backend execution for performance.

Appendix

A. Full Source Code

Value.py -- Core autograd engine

```
from math import exp

class Value:
    def __init__(self, data):
        self.data = data
        self.grad = 0.0
        self._prev = set()
        self._backward = lambda: None # default: no-op (leaf node)

    def __repr__(self):
        return f"Value(data={self.data:.4f}, grad={self.grad:.4f})"

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data)
        out._prev = {self, other}
        def _backward():
            self.grad += 1.0 * out.grad # add: local gradient = 1 (distributor)
            other.grad += 1.0 * out.grad
        out._backward = _backward
        return out

    def __radd__(self, other): # support: 2 + Value
        return self + other

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data)
        out._prev = {self, other}
        def _backward():
            self.grad += other.data * out.grad # mul: local gradient = swap
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def __rmul__(self, other): # support: 2 * Value
        return self * other

    def __neg__(self): # -Value
        return self * -1

    def __sub__(self, other): # Value - other
        return self + (-other)

    def __rsub__(self, other): # other - Value
        return other + (-self)

    def backward(self):
        # 1. topological sort via DFS post-order
        topo = []
        visited = set()
        def build_topo(node):
            if node not in visited:
                visited.add(node)
                for child in node._prev:
                    build_topo(child)
        build_topo(self)
```

```

        build_topo(child)
        topo.append(node)
    build_topo(self)
    # 2. set root gradient = 1 (dL/dL = 1)
    self.grad = 1.0
    # 3. call _backward in reverse topological order
    for node in reversed(topo):
        node._backward()

def __pow__(self, n):
    # n is a constant (int/float)
    out = Value(self.data ** n)
    out._prev = {self}
    def _backward():
        self.grad += (n * self.data ** (n - 1)) * out.grad
    out._backward = _backward
    return out

def relu(self):
    out = Value(max(0, self.data))
    out._prev = {self}
    def _backward():
        self.grad += (1.0 if self.data > 0 else 0.0) * out.grad
    out._backward = _backward
    return out

def sigmoid(self):
    s = 1 / (1 + exp(-self.data))
    out = Value(s)
    out._prev = {self}
    def _backward():
        self.grad += s * (1 - s) * out.grad # sigmoid derivative
    out._backward = _backward
    return out

```

nn.py -- Neural network modules

```

from random import uniform
from Value import Value

class Neuron:
    def __init__(self, nin, activation=True):
        self.w = [Value(uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(0.0)
        self.activation = activation
    def __call__(self, x):
        ws = self.b
        for wi, xi in zip(self.w, x):
            ws = ws + wi * xi
        return ws.relu() if self.activation else ws
    def parameters(self):
        return self.w + [self.b]

class Layer:
    def __init__(self, nin, nout, activation=True):
        self.neurons = [Neuron(nin, activation) for _ in range(nout)]
    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs[0] if len(outs) == 1 else outs
    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]

class MLP:

```

```

def __init__(self, nin, nouts):
    sz = [nin] + nouts
    self.layers = [Layer(sz[i], sz[i+1],
        activation=(i != len(nouts)-1))
        for i in range(len(nouts))]
def __call__(self, x):
    for layer in self.layers:
        x = layer(x)
    return x
def parameters(self):
    return [p for layer in self.layers for p in layer.parameters()]

```

train.py -- XOR training script

```

from Value import Value
from nn import MLP

if __name__ == '__main__':
    # 1. XOR data
    xs = [
        [Value(0), Value(0)],
        [Value(0), Value(1)],
        [Value(1), Value(0)],
        [Value(1), Value(1)],
    ]
    ys = [0, 1, 1, 0]

    # 2. Model: input=2, hidden=[8,4], output=1
    model = MLP(2, [8, 4, 1])

    # 3. Training loop
    lr = 0.05
    epochs = 1000

    for epoch in range(epochs):
        preds = [model(x) for x in xs]          # forward
        loss = Value(0)
        for pred, y in zip(preds, ys):
            loss = loss + (pred - y) ** 2        # MSE loss
        for p in model.parameters():
            p.grad = 0.0                        # zero grad
        loss.backward()                         # backpropagation
        for p in model.parameters():
            p.data -= lr * p.grad                # gradient descent

        if epoch % 10 == 0:
            print(f"epoch {epoch:3d} | loss = {loss.data:.6f}")

    # 4. Final results
    print("\n=== XOR Results ===")
    for x, y in zip(xs, ys):
        pred = model(x)
        print(f"input={[xi.data for xi in x]}, pred={pred.data:.4f}, target={y}")

```

verify.py -- PyTorch comparison

```

import torch
from Value import Value

# Test 1: add, mul, pow
a = Value(2.0); b = Value(3.0)
e = (a * (a + b)) ** 2; e.backward()

```

```

a_t = torch.tensor(2.0, requires_grad=True)
b_t = torch.tensor(3.0, requires_grad=True)
e_t = (a_t * (a_t + b_t)) ** 2; e_t.backward()

assert abs(a.grad - a_t.grad.item()) < 1e-6 # 140.0
assert abs(b.grad - b_t.grad.item()) < 1e-6 # 40.0

# Test 2: relu
x = Value(-3.0); y = Value(4.0)
w = (x + y).relu() * Value(2.0); w.backward()
# x.grad = 2.0, y.grad = 2.0 (all match PyTorch)

# Test 3: sigmoid
a2 = Value(1.5)
c2 = a2.sigmoid() * Value(3.0); c2.backward()
# a2.grad = 0.4474 (matches PyTorch)

# Test 4: multiple path contribution
a3 = Value(3.0); b3 = Value(5.0)
f = a3 * a3 + a3 * b3; f.backward()
# a3.grad = 11.0, b3.grad = 3.0 (all match PyTorch)

```

B. Experimental Results

XOR training was run 3 times with different random seeds. All 3 runs converged to loss ~ 0.0 within 200 epochs using MLP(2, [8, 4, 1]) with $lr=0.05$. Earlier attempts with shallower networks (MLP(2, [4, 1])) occasionally got stuck at local minima (loss ~ 0.667). Deeper networks with small learning rates suffered from dead ReLU (all neurons outputting 0).

C. AI Interaction Log

This project was developed through an interactive session with Claude (Anthropic). The AI served as a tutor, providing: (1) conceptual explanations of computational graph theory, (2) code review and debugging hints (e.g., identifying reversed ReLU gradient condition, `cmath` vs `math` import issue), (3) architectural guidance for the Value/Neuron/Layer/MLP class hierarchy, and (4) PyTorch verification test design. The student wrote the initial code attempts and iteratively refined them based on the AI's feedback.